

PRD Prompt — Whitelabel do Design System Horizon ("DS-Horizon")

O que é este arquivo? Um prompt operacional completo. Cole este arquivo inteiro em uma conversa com um agente de IA (Claude Code, Cursor, etc.) dentro do projeto de destino, junto com os assets da nova marca (logo, cores, fontes), e o agente vai adaptar o pacote `DS-Horizon` (baixado junto com este PRD) para a identidade visual do novo cliente — **sem alterar a estrutura, os nomes de tokens, nem o contrato dos componentes.**

1. Contexto — o que é o DS-Horizon

O DS-Horizon é o design system da plataforma **Horizon** (FAMI/4Lumen), construído sobre:

- **React 18 + TypeScript + Vite**
- **Tailwind CSS** (tokens via CSS variables, `darkMode: ["class"]`)
- **shadcn/ui + Radix UI** (componentes acessíveis, não estilizados)
- **Atomic Design:** Átomos (button, input, badge...) → Moléculas (FilterBar, ClienteForm, MultiTagInput...) → Organismos (AppSidebar, DataTable, BoletadorKanban...)
- **Storybook 10** como catálogo vivo de tokens e componentes

A premissa central: **nenhum componente referencia cor "crua"**. Tudo passa por três camadas de tokens (ver seção 3). Isso é o que torna o whitelabel possível: trocando apenas os **valores** dos tokens de primitivos e os **assets de marca** (logo, favicon), toda a interface se readequa automaticamente — light e dark mode incluídos — sem tocar em nenhum `.tsx` de componente.

2. O que vem no pacote `ds-horizon-package.zip`

```
DS-Horizon/
├─ README.md           ← guia rápido de instalação
├─ components.json      ← config shadcn/ui (aliasas @/components, @/lib, etc.)
├─ tailwind.config.ts   ← mapeamento de tokens semânticos → cores Tailwind
├─ postcss.config.js
├─ src/
│  ├─ index.css         ← AS 3 CAMADAS DE TOKENS (ponto central do whitelabel)
│  ├─ lib/
│  │  └─ utils.ts       ← helper `cn()` (clsx + tailwind-merge)
│  └─ hooks/
```

```

|   └─ use-mobile.tsx
|   └─ use-toast.ts
└─ components/
    └─ ui/                ← ~50 componentes shadcn/ui (átomos)
    └─ BrandLogo.tsx      ← template do componente de logo (ex-HorizonLogo)
    └─ BrandLoadingLogo.tsx ← template do componente de loading com logo

```

Dependências npm necessárias (instalar no projeto de destino — ver `README.md` do pacote para a lista completa de versões): Radix UI (`@radix-ui/react-*`), `class-variance-authority` , `clsx` , `tailwind-merge` , `tailwindcss-animate` , `lucide-react` , `cmdk` , `vaul` , `embla-carousel-react` , `recharts` , `date-fns` , `react-day-picker` , `react-hook-form` , `@hookform/resolvers` , `zod` , `sonner` , `next-themes` , `input-otp` .

3. Arquitetura de tokens (3 camadas) — NÃO RENOMEAR

Toda a customização de marca acontece **dentro de** `src/index.css` e no bloco `colors` de `tailwind.config.ts` . As três camadas:

Camada 1 — Primitivos (`--primitive-*`)

Paletas brutas de 11 tons (50→950), sem semântica. No Horizon são `gold` (marca) e `navy` (estrutural escura), além de `gray` , `green` , `red` , `amber` , `blue` , etc. (estes últimos são neutros/feedback e **geralmente não precisam mudar**).

```

--primitive-gold-500: #BC955C; /* cor primária da marca Horizon */
--primitive-navy-800: #243746; /* cor escura estrutural */

```

Camada 2 — Semânticos (`--semantic-*`)

Dão significado aos primitivos e mudam entre light/dark:

```

--semantic-color-brand: var(--primitive-gold-500);
--semantic-color-brand-hover: var(--primitive-gold-600);
--semantic-color-text-brand: var(--primitive-gold-600);
--semantic-color-border-focus: var(--primitive-gold-500);

```

Camada 3 — Componente (`--comp-*`)

Escopo por componente, referenciam apenas semânticos:

```

--comp-button-primary-bg: var(--semantic-color-brand);
--comp-button-primary-hover: var(--semantic-color-brand-hover);

```

Camada Tailwind (HSL) — @layer base

Os tokens consumidos pelos componentes shadcn (`bg-primary` , `text-foreground` , `border-input` ...) são definidos como **triplos HSL sem unidade** (`H S% L%`), duplicados em `:root` (dark) e `.light` :

```
.light {
  --primary: 36 42% 55%;          /* = #C4A267 em HSL, sem "hsl()" */
  --primary-foreground: 0 0% 100%;
  --ring: 36 42% 55%;
  ...
}
:root {
  --primary: 36 42% 55%;          /* mesma cor de marca também no dark */
  ...
}
```

⚠ **Regra de ouro do whitelabel:** os *nomes* das variáveis (`--primary` , `--primitive-gold-500` , `--semantic-color-brand` , etc.) e os mapeamentos em `tailwind.config.ts` (`primary: { DEFAULT: "hsl(var(--primary))", ... }`) **não mudam**. Apenas os **valores hex/HSL** são substituídos pelos da nova marca. Isso garante que todos os componentes (que usam `bg-primary` , `text-muted-foreground` etc.) continuem funcionando sem qualquer edição.

4. Pontos de customização de marca

O que muda	Onde	Como
Cor primária	<code>src/index.css</code> → paleta <code>--primitive-gold-*</code> (renomear paleta se quiser, ex. <code>--primitive-brand-*</code>) + <code>--primary</code> / <code>--ring</code> / <code>--sidebar-primary</code> (HSL) em <code>.light</code> e <code>:root</code>	Gerar escala de 50→950 a partir da cor primária da nova marca
Cor secundária/estrutural	<code>--primitive-navy-*</code> + <code>--secondary</code> , <code>--accent</code> , <code>--sidebar-accent</code>	Idem, a partir da cor secundária da marca
Cores de feedback (success/warning/error/info)	<code>--success</code> , <code>--warning</code> , <code>--destructive</code> , <code>--info</code>	Manter padrão Horizon, salvo pedido explícito do cliente
Tipografia	<code>tailwind.config.ts</code> → <code>fontFamily.heading</code> / <code>fontFamily.body</code> , e <code>@import</code> do Google Fonts em <code>src/index.css</code>	Trocar família de fonte; manter os nomes <code>font-heading</code> / <code>font-body</code>

O que muda	Onde	Como
Logo (header, sidebar, loading)	<code>src/components/BrandLogo.tsx</code> (ex <code>HorizonLogo.tsx</code>) e <code>BrandLoadingLogo.tsx</code>	Substituir os <code><path></code> do SVG pelo logo da nova marca, preservando as props <code>variant</code> , <code>scheme</code> , <code>height</code>
Favicon / imagens públicas	<code>public/favicon.svg</code> , <code>public/Logo-*.svg</code> , <code>public/horizon-logo-*.svg</code>	Substituir pelos arquivos da nova marca mantendo os mesmos nomes referenciados no código, ou atualizar as referências
Raio de borda / sombras / espaçamento	<code>--primitive-radius-*</code> , <code>--primitive-shadow-*</code> , <code>--primitive-space-*</code>	Só mudar se a marca pedir um estilo mais quadrado/arredondado

5. Passo a passo operacional (para a IA executar)

Quando este prompt for colado em um novo projeto, junto com os dados da marca (nome, paleta de cores primária/secundária/terciária em hex, arquivo(s) de logo em SVG, e opcionalmente fontes), execute nesta ordem:

- 1. Extrair o pacote** `DS-Horizon/` na raiz do projeto de destino, mesclando as pastas `src/components/ui`, `src/lib`, `src/hooks` e copiando `src/index.css`, `tailwind.config.ts`, `components.json`, `postcss.config.js` (ajustando paths de import `@/` conforme o `tsconfig` / `vite.config` do projeto de destino).
- 2. Instalar dependências** listadas no `README.md` do pacote (via `npm install`).
- 3. Gerar a escala primitiva da nova marca:** a partir do hex primário e secundário fornecidos, gerar paletas de 11 tons (50→950) seguindo o mesmo critério de luminância das paletas `gold` / `navy` originais. Substituir os valores de `--primitive-gold-*` e `--primitive-navy-*` (ou renomear a paleta para o nome da marca, ajustando as referências nos tokens semânticos).
- 4. Atualizar tokens semânticos e de componente** (`--semantic-color-brand*`, `--comp-button-*`, etc.) para apontar para a nova paleta primitiva — apenas troca de referência, sem mudar nomes.
- 5. Atualizar a camada HSL** (`--primary`, `--ring`, `--sidebar-primary`, `--secondary`, `--accent`, `--sidebar-accent`, etc.) em `:root` e `.light` com os mesmos tons convertidos para HSL `H S% L%`.
- 6. Substituir o logo:** editar `BrandLogo.tsx` / `BrandLoadingLogo.tsx` trocando os `<path d=" ... " fill=" ... ">` pelo SVG da nova marca (versões `light` / `dark` / `icon`), preservando a assinatura do componente (`variant`, `scheme`, `className`, `height`) para não quebrar os usos existentes (`AppHeader`, `AppSidebar`, telas de loading/login).

7. **Tipografia (opcional)**: se a marca tiver fontes próprias, atualizar o `@import` no topo de `src/index.css` e `fontFamily.heading / body` em `tailwind.config.ts`. Caso contrário, manter Inter/Montserrat.
 8. **Validar** levantando o Storybook (`npm run storybook`) e revisando as páginas `Design System / Tokens / Cores`, `Tipografia` e os componentes `Button`, `Badge`, `Card` em light e dark — todas as cores devem refletir a nova marca automaticamente.
 9. **Não criar novos componentes nem renomear arquivos de** `src/components/ui` — o objetivo é reuso 1:1 da biblioteca, só a casca visual muda.
-

6. Restrições (NUNCA fazer)

- ❌ Não renomear variáveis CSS (`--primary`, `--primitive-*`, `--semantic-*`, `--comp-*`) nem chaves do `tailwind.config.ts` (`colors.primary`, `colors.sidebar`, etc.).
 - ❌ Não editar a lógica/markup dos componentes em `src/components/ui/*` para "encaixar" uma cor — sempre resolva via token.
 - ❌ Não misturar cor "crua" (`#hex` direto em `className` ou inline style) em componentes de produto — sempre `bg-primary`, `text-muted-foreground`, etc.
 - ❌ Não remover o suporte a dark mode (`:root` vs `.light`) ao trocar valores.
-

7. Checklist de aceite do whitelabel

- ☐ Paleta primária/secundária da nova marca aplicada nas 3 camadas de tokens (primitivo, semântico, componente) + camada HSL Tailwind
 - ☐ Logo da nova marca renderiza corretamente em `variant="icon"` e `variant="full"`, em `scheme="light"` e `scheme="dark"`
 - ☐ Light mode e dark mode revisados (cores de marca presentes em ambos)
 - ☐ `Button`, `Badge`, `Card`, `Sidebar`, `Tabs` revisados no Storybook com a nova paleta
 - ☐ Nenhum nome de token/variável foi alterado — apenas valores e assets
 - ☐ Build (`npm run build`) e Storybook (`npm run storybook`) sem erros
-

8. Dados de marca a serem fornecidos pelo usuário (input deste prompt)

Preencha antes de executar o passo a passo:

Nome da marca: _____
Cor primária (hex): _____
Cor secundária (hex): _____
Cor terciária/acento (hex, opcional): _____
Logo – versão light (SVG): _____
Logo – versão dark (SVG): _____
Logo – ícone isolado (SVG, opcional): _____
Fonte de heading (opcional, default Montserrat): _____
Fonte de body (opcional, default Inter): _____

Gerado a partir do Design System Horizon (v0.6.0) — este documento e o pacote `ds-horizon-package.zip` são distribuídos juntos pela página "Boas-vindas" do Storybook do projeto Horizon.